

Module 9: Generative AI, Spring AI, Embeddings, RAG, Advisors, and Tool Calling

Questions

Q1. What is Generative AI, and how does it differ from traditional AI?

Answer: Generative AI refers to models that can create new content such as text, images, audio, or code by learning patterns from training data. Unlike traditional AI, which is often used for classification, regression, or clustering, generative AI produces outputs that resemble its training distribution. Large language models such as GPT, Gemini, and Claude are generative AI systems trained to predict the next token in a sequence. For backend developers, generative AI enables features like chatbots, code assistants, content generation, and summarization. The key difference is that traditional AI maps inputs to known labels, while generative AI synthesizes novel outputs conditioned on inputs.

Q2. What is a Large Language Model, and how does it work at a high level?

Answer: A Large Language Model is a deep neural network, typically based on the Transformer architecture, trained on vast amounts of text to predict the next token in a sequence. During training, the model learns statistical relationships between words, grammar, facts, and reasoning patterns. At inference, it takes a prompt, converts it into tokens, processes it through many layers of self-attention and feed-forward networks, and generates tokens one at a time until a stop condition is reached. The model does not truly understand text in a human sense; it predicts likely sequences based on learned patterns. Modern LLMs have billions of parameters and are fine-tuned for instruction following and safety.

Q3. What is tokenization, and why does it matter?

Answer: Tokenization is the process of splitting text into smaller units called tokens, which can be words, subwords, or characters. LLMs operate on tokens rather than raw characters, so the input text is first converted to tokens, and the model predicts the next token. The number of tokens affects cost, context window usage, and latency. Different models use different tokenizers, so the same sentence may produce different token counts across providers. For backend developers, tokenization matters

because APIs charge per token, prompt length is limited by the context window, and output must often be bounded by max tokens.

Q4. What is the Transformer architecture?

Answer: The Transformer is a neural network architecture introduced in the paper Attention Is All You Need. It is built from encoder and decoder stacks, though modern LLMs typically use decoder-only variants. The core component is the self-attention mechanism, which allows the model to weigh the importance of different tokens in the input when producing each output token. Transformers also use positional encoding to understand token order, feed-forward layers to transform representations, layer normalization, and residual connections. The architecture is highly parallelizable during training and scales well to billions of parameters.

Q5. What is self-attention, and why is it important?

Answer: Self-attention is a mechanism that computes a weighted representation of each token by considering all other tokens in the sequence. For every token, the model generates query, key, and value vectors, compares the query with all keys to produce attention scores, normalizes them with softmax, and uses the weighted sum of values as the output. This allows the model to capture long-range dependencies and contextual relationships regardless of token distance. It is important because it replaces sequential processing with parallel computation and gives the model the ability to focus on relevant parts of the input when generating each token.

Q6. What are embeddings in the context of generative AI?

Answer: Embeddings are dense numerical vector representations of text, images, or other data. They map semantic meaning into a high-dimensional space where similar items are closer together. In LLMs, token embeddings represent individual tokens, while sentence or document embeddings capture the meaning of larger text chunks. Embeddings are produced by passing text through an embedding model. They are used for tasks such as semantic search, classification, clustering, recommendation, and retrieval in RAG systems. The quality of embeddings depends on the model and the domain of the training data.

Q7. What is a vector store and vector search?

Answer: A vector store is a database optimized for storing and searching high-dimensional vectors. Vector search, also called similarity search, finds vectors that are closest to a query vector using distance metrics such as cosine similarity or Euclidean distance. Popular vector stores include

pgvector, Pinecone, Weaviate, Chroma, Milvus, and Redis. They often use approximate nearest neighbor algorithms such as HNSW or IVF to search efficiently at scale. In generative AI applications, vector stores are used to retrieve relevant documents or facts based on semantic similarity, which improves response accuracy beyond what the model memorized during training.

Q8. What is Retrieval Augmented Generation?

Answer: Retrieval Augmented Generation is a pattern that combines information retrieval with text generation. Instead of relying solely on the model's training knowledge, the system retrieves relevant documents from a vector store or database and includes them in the prompt as context. The model then generates an answer grounded in the retrieved content. RAG reduces hallucination, keeps responses up to date, and allows the model to answer questions about private or proprietary data. It is commonly used for enterprise knowledge bases, documentation assistants, and customer support bots.

Q9. What is the difference between naive RAG and modular RAG?

Answer: Naive RAG embeds the user query, performs a single similarity search against a vector store, and injects the top documents into the prompt. It is simple to implement and works well for many cases. Modular RAG breaks the process into configurable stages such as query transformation, retrieval, document post-processing, and augmentation. It can use query rewriting, compression, translation, multi-query expansion, reranking, and hybrid search. Modular RAG is more flexible and accurate but also more complex. Spring AI supports naive RAG through `QuestionAnswerAdvisor` and modular RAG through `RetrievalAugmentationAdvisor`.

Q10. What is Spring AI?

Answer: Spring AI is a framework that brings AI capabilities to Spring applications. It provides a consistent, portable API for working with large language models, embedding models, vector stores, and AI patterns such as RAG and tool calling. It supports multiple model providers including OpenAI, Anthropic, Google Gemini, Ollama, Azure OpenAI, and Mistral. Spring AI is designed with the same philosophy as Spring: abstractions over vendor APIs, easy configuration, and integration with Spring Boot. Backend developers can use it to add chat, search, classification, and agentic features to Java applications without writing provider-specific boilerplate.

Q11. What is ChatClient in Spring AI?

Answer: ChatClient is the high-level fluent API in Spring AI for interacting with chat models. It allows developers to build prompts, set model options, attach advisors, register tools, and call or stream responses. It is constructed from a ChatModel bean and configured through the builder. ChatClient abstracts the underlying provider, so switching from OpenAI to Anthropic requires changing configuration rather than application code. It supports synchronous calls with `call()`, streaming with `stream()`, structured output with `entity()`, and content extraction with `content()`.

Q12. How do you create a ChatClient in Spring Boot?

Answer: You add the appropriate Spring AI starter, such as `spring-ai-openai-spring-boot-starter`, and configure properties like `spring.ai.openai.api-key` and `spring.ai.openai.chat.options.model`. Spring Boot auto-configures a ChatModel bean. You then create a ChatClient bean by calling `ChatClient.builder(chatModel).build()`. You can also set default advisors, default tools, and default options during construction. The ChatClient can be injected into services and used in REST controllers or background jobs. This pattern keeps AI logic inside Spring-managed services and integrates with the rest of the application.

Q13. What are Advisors in Spring AI?

Answer: Advisors are interceptors in the ChatClient request and response pipeline. They can modify the request before it reaches the model, process the response after it returns, or implement reusable patterns such as logging, chat memory, RAG retrieval, and tool calling. Advisors implement `CallAdvisor` or `StreamAdvisor` and run in a chain ordered by `getOrder`. Built-in advisors include `MessageChatMemoryAdvisor` for conversation memory, `QuestionAnswerAdvisor` for RAG, `ToolCallingAdvisor` for tool execution, and `RetrievalAugmentationAdvisor` for modular RAG. Custom advisors allow developers to add cross-cutting concerns like audit logging, prompt injection prevention, or cost tracking.

Q14. How do Advisors compare to Spring AOP interceptors?

Answer: Both Advisors and Spring AOP interceptors wrap behavior around a core operation. AOP intercepts method calls in the Spring application context, while Spring AI Advisors intercept calls to the language model. AOP is general purpose for business methods; Advisors are specialized for the ChatClient pipeline. In Spring AI, advisors receive `ChatClientRequest` and `ChatClientResponse` objects and can update context, modify prompts, or transform responses. They are specific to AI workflows and integrate with the ChatClient builder, whereas AOP interceptors work with any Spring bean method.

Q15. What is chat memory, and how is it managed in Spring AI?

Answer: Chat memory stores the history of a conversation so the model can maintain context across multiple turns. Without memory, each request is stateless and the model does not remember previous messages. Spring AI provides `MessageChatMemoryAdvisor`, which stores messages in an in-memory or persistent `ChatMemory` implementation. The advisor retrieves recent messages and adds them to the prompt. For longer conversations, `VectorStoreChatMemoryAdvisor` can retrieve semantically relevant past messages from a vector store. A conversation ID is used to scope messages to a specific user session.

Q16. What is Tool Calling in Spring AI?

Answer: Tool calling, also known as function calling, allows an LLM to invoke application methods to retrieve data or perform actions. In Spring AI, tools are defined by methods annotated with `@Tool` or by implementing `ToolCallback`. The `ChatClient` registers tools, and the `ToolCallingAdvisor` manages the loop: the model decides to call a tool, Spring AI executes the method, and the result is sent back to the model. The loop continues until the model produces a final response. Tool calling enables agents that can access real-time data, query databases, call external APIs, or take actions in the system.

Q17. How do you define a tool in Spring AI?

Answer: You create a method annotated with `@Tool` inside a Spring bean and pass the bean to `ChatClient.defaultTools`. The method should have a descriptive name and parameters, optionally with `@ToolParameter` annotations to describe them to the model. When the model needs the capability, it requests the tool with JSON arguments. Spring AI deserializes the arguments, invokes the method, and serializes the return value back to the model. Tools should be idempotent when possible and avoid side effects that cannot be undone. Complex tools may also use a `ToolCallback` implementation for full control over behavior and metadata.

Q18. What is dynamic tool discovery in Spring AI?

Answer: Dynamic tool discovery is a pattern for applications with many tools. Instead of sending all tool definitions to the model on every request, which consumes tokens and may confuse the model, a `ToolSearchToolCallingAdvisor` indexes tools and exposes a search tool. The model calls the search tool to find relevant tools, and only those tools are added to the context. This reduces token usage and improves accuracy when the application has dozens or hundreds of tools. Tool indexes can be semantic using `VectorToolIndex`, keyword-based using `LuceneToolIndex`, or pattern-based using `RegexToolIndex`.

Q19. What is structured output in Spring AI?

Answer: Structured output is the ability to ask an LLM to return data in a specific format, usually JSON, and map it to a Java object. Spring AI supports this through the `entity()` method on `ChatClient`, which parses the response into a POJO using a schema or instructions provided in the prompt. It can also use JSON mode or constrained decoding if the model supports it. Structured output is useful when you need to consume AI output programmatically, such as extracting entities, classifying text, or generating configuration objects. It is more reliable than parsing free-form text.

Q20. What are query transformers in modular RAG?

Answer: Query transformers rewrite or expand the user query before retrieval to improve search quality. Examples include `RewriteQueryTransformer`, which reformulates the query; `CompressionQueryTransformer`, which shortens conversational queries; `TranslationQueryTransformer`, which translates the query into another language; and `MultiQueryExpander`, which generates multiple related queries to broaden retrieval. These transformers help when user queries are ambiguous, conversational, or domain-specific. They are part of the `RetrievalAugmentationAdvisor` pipeline in Spring AI.

Q21. What are common vector stores used with Spring AI?

Answer: Spring AI supports several vector stores including `pgvector`, Pinecone, Weaviate, Chroma, Milvus, Redis, Neo4j, and Elasticsearch. The choice depends on existing infrastructure, scale, and query requirements. `pgvector` is convenient when the application already uses PostgreSQL. Pinecone and Weaviate are managed services optimized for vector search. Redis is useful when the application already uses Redis for caching. Milvus and Elasticsearch are suitable for large-scale or hybrid search scenarios. Each has a Spring Boot starter with auto-configuration.

Q22. How do you ingest documents into a vector store for RAG?

Answer: Ingestion typically involves reading documents from files or APIs, splitting them into chunks, computing embeddings for each chunk using an `EmbeddingModel`, and storing the resulting vectors and metadata in a `VectorStore`. Chunking strategy matters: too large chunks dilute relevance, while too small chunks lose context. Spring AI provides document readers, transformers, and splitters such as `TokenTextSplitter` and `PagePdfDocumentReader`. Metadata such as source, page number, or document type is stored alongside the vector to enable filtering. After ingestion, RAG queries can retrieve the most relevant chunks.

Q23. What are hallucinations, and how can RAG reduce them?

Answer: Hallucinations are cases where a language model generates plausible-sounding but incorrect or unsupported information. They happen because models are trained to predict likely tokens, not to verify facts against a knowledge source. RAG reduces hallucinations by grounding the model's response in retrieved documents. The prompt explicitly instructs the model to answer based only on the provided context. If the context does not contain the answer, the model can be instructed to say it does not know. Combining RAG with citations, reranking, and strict prompt instructions further improves factual reliability.

Q24. What are prompt templates, and why are they used?

Answer: A prompt template is a reusable text pattern with placeholders that are filled at runtime. It allows developers to separate prompt engineering from application code and to inject dynamic values such as user input, retrieved context, or conversation history. Spring AI provides PromptTemplate and a fluent prompt builder. Templates improve maintainability because prompts can be adjusted without changing Java code. They also support system messages, user messages, and assistant messages, which help structure the conversation and guide model behavior.

Scenario based questions

S1. A Spring AI chatbot gives outdated answers about company policy. How do you fix it?

Answer: The chatbot is relying on the model's training knowledge, which has a cutoff and may not include the latest company documents. I implement RAG by ingesting the company policy documents into a vector store. I split the documents into chunks, compute embeddings, and store them with metadata such as document name and effective date. I add a QuestionAnswerAdvisor to the ChatClient so each user query retrieves the most relevant policy chunks and includes them in the prompt. I instruct the model to answer only from the provided context and to say it does not know if the answer is not in the documents. I also schedule periodic re-ingestion when policies are updated. Real-time example: an HR chatbot tells users the old remote-work policy. I create a pipeline that loads the latest policy PDFs, splits them into 500-token chunks, embeds them with the default embedding model, and stores them in pgvector. The ChatClient uses QuestionAnswerAdvisor with a filter on the latest policy version. Responses now reflect current rules.

S2. A RAG application retrieves irrelevant documents. What do you improve?

Answer: I investigate the retrieval pipeline end to end. I check chunk size and overlap; chunks that are too large dilute relevance, and chunks that are too small lose context. I try different embedding models fine-tuned for the domain. I add query transformation such as query rewriting or multi-query expansion before retrieval. I implement hybrid search combining vector similarity with keyword matching. I add a reranking step using a cross-encoder model that scores retrieved documents against the query. I also verify metadata filters are correct and that the vector store is populated with clean, representative documents. Real-time example: a legal assistant retrieves contracts instead of clauses. I reduce chunk size to 512 tokens with 128 overlap, switch to a domain-specific embedding model, add a keyword boost for clause titles, and use a reranker. Precision improves from 40 percent to 85 percent.

S3. A chatbot does not remember the previous question. How do you add memory?

Answer: I add a chat memory advisor to the ChatClient. I configure MessageChatMemoryAdvisor with a ChatMemory implementation such as InMemoryChatMemory or a persistent store. I pass a conversation ID with each request so the advisor can retrieve and store messages for that session. For long conversations, I use VectorStoreChatMemoryAdvisor to retrieve semantically relevant past turns instead of sending the entire history. I also set a maximum history length to avoid exceeding the model's context window. Real-time example: a customer support bot forgets the user's order ID after one turn. I add MessageChatMemoryAdvisor to the ChatClient, generate a session ID when the conversation starts, and pass it in the advisors parameter. The bot now references earlier messages and asks clarifying questions only when needed.

S4. A Spring AI application needs to answer questions about live order status. How do you design it?

Answer: I use tool calling to fetch live data. I define a @Tool method such as getOrderStatus(String orderId) that queries the order database or order service. I register the tool with ChatClient.defaultTools. When the user asks about an order, the model calls the tool with the order ID, the application returns the current status, and the model answers in natural language. I combine this with RAG for general policy or product questions. I also add validation to ensure users can only query their own orders, which prevents unauthorized data access. Real-time example: a user asks, Where is my order 12345? The ChatClient recognizes it needs order status, calls getOrderStatus("12345"), which returns [{"status": "shipped", "eta": "2025-08-10"}], and the model responds: Your order 12345 has been shipped and is expected to arrive on August 10.

S5. A model's tool call fails because arguments are malformed. How do you debug it?

Answer: I inspect the chat response to see the exact tool name and arguments the model produced. Malformed arguments often happen when the tool description or parameter names are ambiguous. I improve the @Tool method name and add @ToolParameter descriptions so the model understands the expected format. I validate tool inputs and return clear error messages so the model can retry with corrected arguments. I also log every tool invocation and response for debugging. If the issue persists, I switch to a more capable model or use constrained output for critical tools. Real-time example: a weather tool receives dates in different formats. I update the @ToolParameter description to say ISO-8601 date only, add input validation, and return an error like Invalid date format, expected YYYY-MM-DD when the format is wrong. The model corrects itself on the next call.

S6. An AI application exposes sensitive internal tools to the model. How do you secure it?

Answer: I apply the principle of least privilege to tool registration. Only tools that are necessary for the use case are registered with ChatClient. Tools that perform destructive or sensitive operations require authorization checks inside the tool method, verifying the user's identity and permissions. I avoid registering admin-only tools on public-facing chatbots. I also add a SafeGuardAdvisor or custom advisor to review tool calls and block dangerous operations. Tools should be idempotent or reversible when possible, and all invocations should be logged for audit. Real-time example: a support chatbot could accidentally call a deleteUserAccount tool. I remove that tool from the public ChatClient, wrap remaining tools with permission checks that verify the caller is an authorized admin, and log every tool call with user ID and arguments.

S7. A Spring AI service streams responses to a frontend but the output is choppy. What do you check?

Answer: I check whether the issue is on the model side or the network side. Streaming from the model returns tokens as they are generated, so latency is expected. Choppy output may be caused by buffering in the web layer or frontend. I ensure the Spring controller returns a Flux and does not collect the entire response before sending. I verify the client consumes the stream correctly with Server-Sent Events or WebFlux. I also check model parameters; some models or configurations buffer more than others. If the backend is buffering, I return the Flux directly and avoid calling block(). Real-time example: a React frontend receives the entire response at once. The Spring controller returns ResponseEntity by calling content(). I change it to return Flux from chatClient.prompt().user(question).stream().content(), and the frontend now shows typing in real time.

S8. A RAG pipeline returns too many chunks and exceeds the context window. How do you limit it?

Answer: I reduce the number of top-K documents retrieved, typically to 3 or 5. I also reduce chunk size so more chunks fit within the context window. I add a compression step using a query compressor to remove redundant content. I use a reranker to select only the most relevant chunks before injecting them into the prompt. I can also use a contextual query augments that summarizes or filters chunks. If the answer still needs more context, I implement a multi-turn strategy where the model asks follow-up questions to refine retrieval. Real-time example: a technical documentation assistant retrieves 10 large chunks and overflows the context window. I set topK to 4, reduce chunk size to 300 tokens, and add a reranker. The prompt now fits and the answers remain accurate.

S9. A Spring AI application switches from OpenAI to Azure OpenAI. What changes are needed?

Answer: Because Spring AI abstracts the model provider, the main change is dependency and configuration. I replace the OpenAI starter with the Azure OpenAI starter and update properties such as `spring.ai.azure.openai.api-key`, `endpoint`, and `deployment-name`. The `ChatClient` code, advisors, and tools usually remain unchanged. I verify that model capabilities such as JSON mode or tool calling are supported by the chosen Azure deployment. I also update environment-specific configuration without changing the application logic. This portability is one of the main benefits of using Spring AI. Real-time example: a company moves from OpenAI to Azure OpenAI for enterprise compliance. I change the Maven dependency, update `application.yml` with the Azure endpoint and deployment name, and rerun tests. The same service now calls Azure without any Java code changes.

S10. A custom advisor in Spring AI needs to log all prompts and responses. How do you implement it?

Answer: I create a class that implements `CallAdvisor` or `StreamAdvisor`, depending on whether I need to intercept synchronous or streaming calls. In the `around` method, I access the `ChatClientRequest`, log the prompt text and metadata, proceed to the next advisor or model, receive the `ChatClientResponse`, and log the response content. I set an order value so the logging advisor runs early in the request chain and late in the response chain. I register it with the `ChatClient` builder as a default advisor. I avoid logging sensitive user data unless required and ensure logs comply with privacy policies. Real-time example: a compliance team requires an audit trail of AI interactions. I implement a `LoggingAdvisor` that writes prompt and response to SLF4J with conversation ID. I set order to `LOWEST_PRECEDENCE` so it wraps the entire chain, and I register it as a default advisor. Audit logs now contain request and response for every AI call.

S11. An application uses RAG but users get different answers for the same question. How do you stabilize it?

Answer: Variability can come from the embedding model, the retrieval algorithm, the LLM, or the prompt. I fix retrieval by setting deterministic parameters such as a fixed top-K and a consistent similarity threshold. I use a reranker to make scoring more stable. I use a deterministic LLM with temperature set to 0 when consistency is more important than creativity. I version the prompt template and the document corpus so the same query produces the same answer against the same data version. I also add citations so users can verify the source. Real-time example: a support bot gives inconsistent refund policy answers. I set topK=5, temperature=0, add a reranker, and pin the embedding model version. I also include citations in the response. The same question now returns nearly identical answers.

S12. A tool-calling agent calls the same tool repeatedly in a loop. How do you stop it?

Answer: I configure a maximum number of tool call iterations on the ToolCallingAdvisor or in application logic. I improve tool descriptions so the model understands when to stop. I add a guard advisor that detects loops and interrupts them. I ensure tools return concise, complete answers rather than partial data that forces repeated calls. For complex workflows, I break the task into explicit steps instead of relying on the model to coordinate everything through tools. Real-time example: an agent calling searchProduct repeatedly with slightly different keywords. I add a loop limit of 5, update the tool description to say call only once and refine the query internally, and add an advisor that aborts if the same tool is called with the same arguments twice. The loop stops.

S13. A vector search returns results but the model ignores them and answers from its training data. How do you fix it?

Answer: I improve the system prompt to explicitly instruct the model to answer only from the provided context and to say it does not know if the answer is not in the context. I make the retrieved context prominent in the prompt, usually by placing it before the user question. I use a clear delimiter such as "Context:" and "Question:" to help the model distinguish sources. I also reduce the temperature so the model is less creative and more grounded. If the model still ignores context, I switch to a model that follows instructions better or fine-tune with examples of grounded answers. Real-time example: a medical assistant answers with generic advice despite retrieved clinical guidelines. I update the system message to "You are a clinical assistant. Answer strictly from the provided guidelines. If the answer is not in the context, say 'I do not have that information.'" and place the context first. The answers now stick to the retrieved documents.

S14. A Spring AI application must handle high request volume. What do you optimize?

Answer: I use streaming to reduce time-to-first-byte and avoid blocking threads. I configure connection pooling for the model provider HTTP client. I cache similar requests where appropriate. I add rate limiting and backpressure to prevent overwhelming the provider API. For RAG, I precompute embeddings and keep the vector store query fast. I use smaller models for simple tasks and reserve large models for complex reasoning. I also consider running a local model such as Ollama for non-critical traffic to reduce latency and cost. Horizontal scaling of the Spring Boot service behind a load balancer handles concurrent users. Real-time example: a content generation API receives 1000 requests per minute. I enable streaming responses, add a 1000-request-per-minute rate limiter, cache prompts with deterministic parameters, and scale the service to five replicas. Latency drops and costs remain controlled.

S15. A chatbot needs to switch language based on the user's message.

How do you implement it?

Answer: I can use a TranslationQueryTransformer before retrieval if the knowledge base is in one language. If the model itself supports multilingual responses, I detect the language from the user message, add a system instruction to respond in that language, and proceed normally. For RAG, I either translate the query to the document language before retrieval or retrieve documents in the detected language using metadata filters. Spring AI advisors make it easy to add a translation step in the modular RAG pipeline. Real-time example: users ask questions in English, Spanish, and German. I add a language detection step, append "Respond in the same language as the user" to the system prompt, and translate the query to English before vector search because the knowledge base is in English. The model answers in the user's language using retrieved English content.

S16. An advisor needs to add user identity context to every prompt. How do you write it?

Answer: I implement a CallAdvisor and StreamAdvisor that retrieves the current user identity from the security context and injects it into the prompt or context map. In the advise method, I access the ChatClientRequest, extract the user principal, and either add a system message with user details or update the advisor context with user attributes. The next advisors and the model can use this context. I register the advisor early in the chain so subsequent advisors such as RAG or tool calling can also use the identity. I ensure sensitive user data is handled according to privacy requirements. Real-time example: an enterprise assistant needs to know the user's department to filter documents. I write a UserContextAdvisor that reads the authenticated user from Spring Security, adds a context parameter "department=engineering", and a later RAG advisor uses that parameter to filter vector search results. Responses are personalized and scoped.

S17. A RAG system needs to cite sources in its answers. How do you add citations?

Answer: I include metadata such as source filename, page number, and URL in the retrieved chunks. In the prompt, I instruct the model to cite the source after each claim using the metadata provided. I use a structured prompt template that includes the context chunks with identifiers such as [1], [2], and asks the model to reference those identifiers. The post-processing step can map identifiers back to full source URLs and display them to the user. Spring AI's document metadata flows through the advisors, so the model has access to the citation fields. Real-time example: a documentation assistant needs to show source links. Each chunk has metadata {"source":"/docs/spring-ai.md","page":42}. The prompt includes these identifiers. The model answers: "Spring AI supports multiple model providers [1]." The frontend maps [1] to the URL and displays it as a clickable citation.

S18. A generative AI feature returns nonsensical output under load. What could be wrong?

Answer: Under load, the issue may be context window overflow due to long prompts, concurrency limits on the model provider, thread exhaustion in the application, or cache inconsistency. I check the prompt size and reduce retrieved context or chat history. I monitor provider rate limits and errors. I verify that the Spring Boot application uses reactive or properly pooled resources and does not block event loops. I also check whether the model temperature or sampling parameters change behavior at scale if requests are batched differently. I add retries with backoff for transient provider errors and circuit breakers for provider outages. Real-time example: during a product launch, an AI summary endpoint returns garbled text. Investigation shows the vector store query times out under load and returns empty results, so the model answers without context. I add a cache for common queries, scale the vector store, and add a fallback that returns a default message instead of calling the model with no context.

S19. A Spring AI application needs to evaluate the quality of generated answers. How do you design evaluation?

Answer: I implement an evaluation pipeline that compares generated answers against reference answers or ground truth. Metrics include relevance, faithfulness to retrieved context, answer correctness, and hallucination detection. I use LLM-based evaluators that score responses with structured output. I also collect user feedback with thumbs up or thumbs down and store it for analysis. I run evaluations on a held-out test set before deploying prompt or model changes. Spring AI provides evaluation utilities, and custom advisors can log inputs and outputs for offline analysis. Real-time example: a legal Q&A system needs to verify answer quality. I build a batch evaluator that sends 500 known questions to the chatbot, compares answers to reference answers using an LLM judge, and

reports precision, recall, and faithfulness scores. Changes that drop scores below a threshold are blocked from deployment.

S20. A company wants to use a local open-source model instead of a cloud provider. How do you do it with Spring AI?

Answer: I use Ollama or a similar local model server. I add the `spring-ai-ollama-spring-boot-starter` dependency and configure `spring.ai.ollama.base-url` and `spring.ai.ollama.chat.options.model`. Spring AI abstracts the provider, so `ChatClient` code remains the same. The local model runs on company infrastructure, keeping data on premises. This is useful for regulated data or cost control. The trade-off is that local models may be less capable than large cloud models, so they work best for simpler tasks or as a fallback. I also ensure the hardware has sufficient GPU or CPU resources to run the model efficiently. Real-time example: a healthcare company cannot send patient data to cloud LLMs. I deploy a Llama model with Ollama on an on-premise GPU server, configure Spring AI to point to the local Ollama endpoint, and use the same RAG and tool-calling advisors. Patient data never leaves the data center.